



北京交通大学
BEIJING JIAOTONG UNIVERSITY



计算机科学与技术学院
School of Computer Science & Technology

第七章 指令系统

尹辉 刘万成

hyin@bjtu.edu.cn

wchliu@bjtu.edu.cn

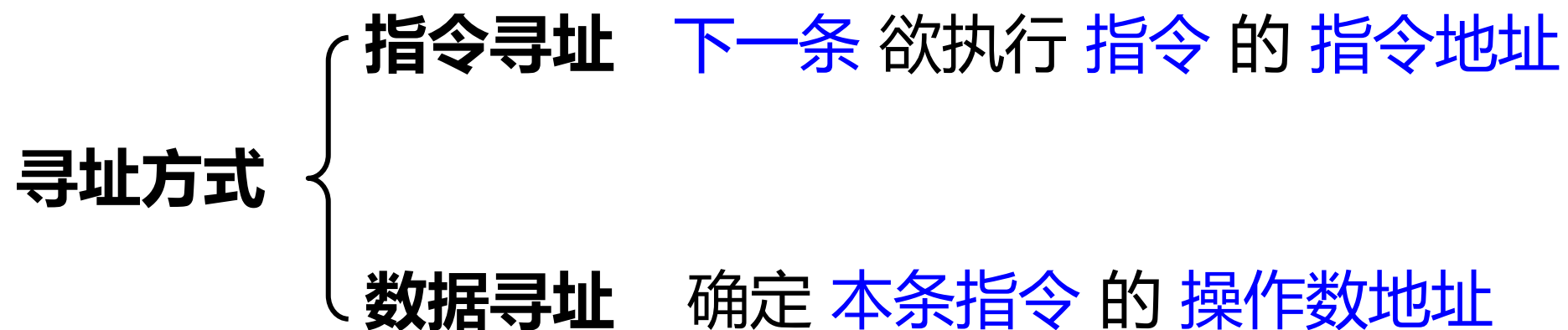
办公地点：九教北312, 401室

- 指令集架构的组成

- 指令的寻址方式

- CISC VS RISC

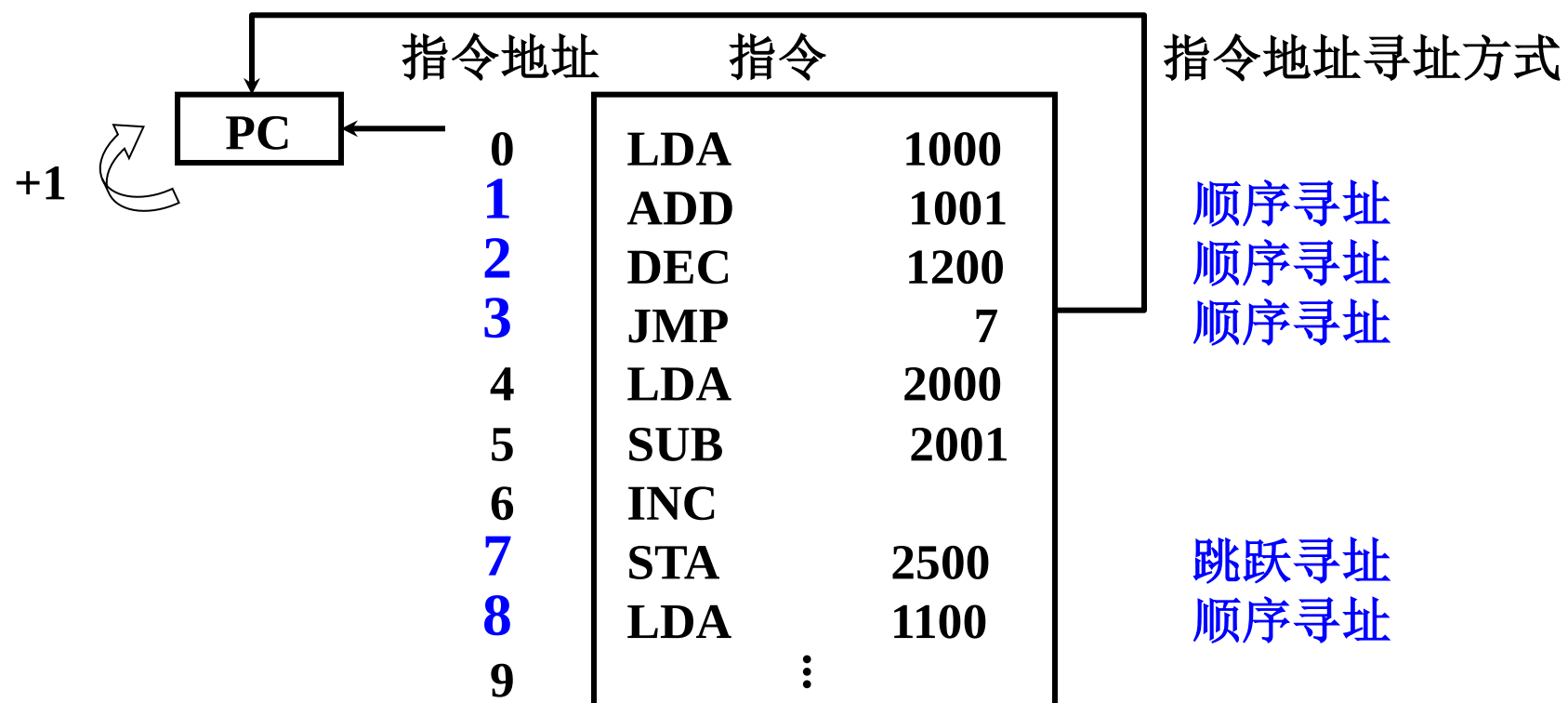
□ 寻址方式



□ 指令寻址

顺序 $(PC) + 1 \longrightarrow PC$

跳跃 由转移指令指出



□ 数据寻址

操作码	寻址特征	形式地址 A
-----	------	--------

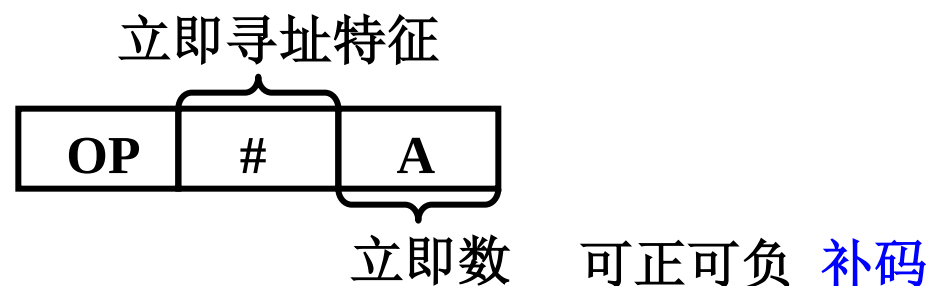
形式地址 指令字中的地址

有效地址 操作数的真实地址

约定 指令字长 = 存储字长 = 机器字长

1. 立即寻址

形式地址 A 就是操作数

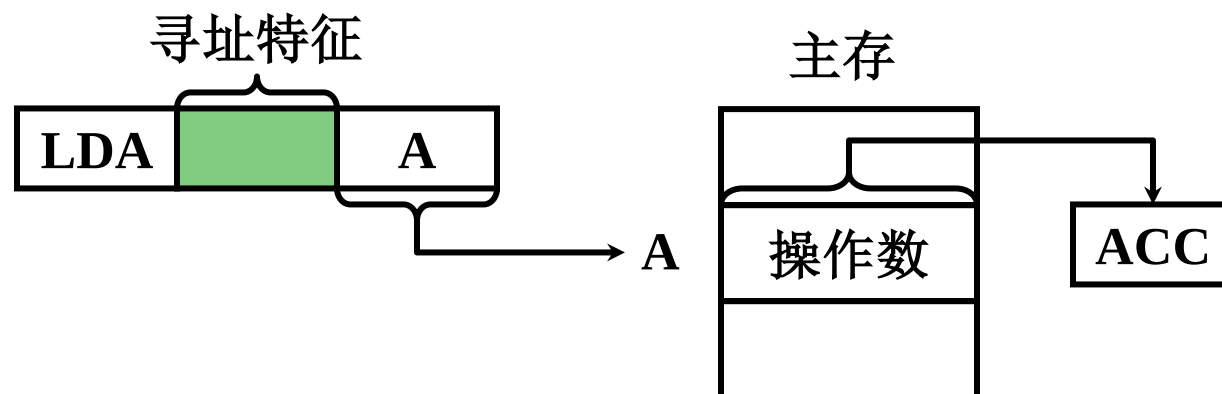


- 指令执行阶段不访存
- A 的位数限制了立即数的范围

□ 数据寻址

2. 直接寻址

$EA = A$ 有效地址由形式地址直接给出



- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围

STRA [2000], R2

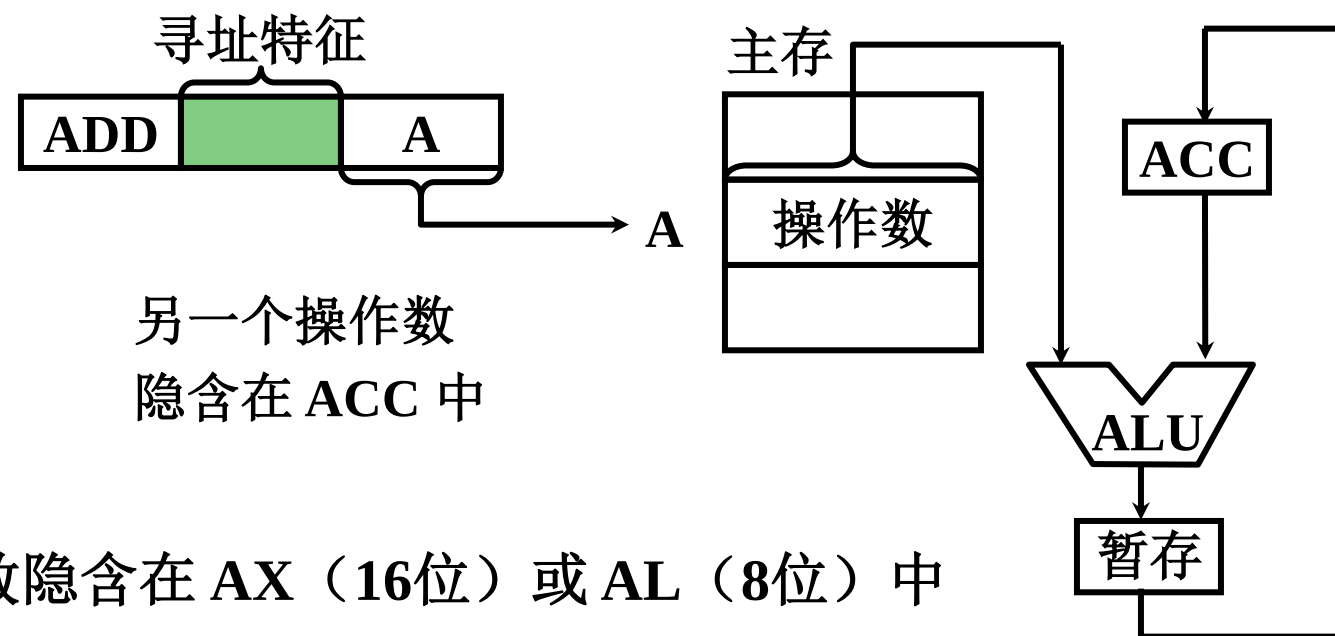
把R2的内容写入到地址为2000的内存单元之中，

A 表示直接地址寻址。

□ 数据寻址

3. 隐含寻址

操作数地址隐含在操作码中



如 8086

MUL 指令 被乘数隐含在 AX (16位) 或 AL (8位) 中

MOVS 指令 源操作数的地址隐含在 SI 中

目的操作数的地址隐含在 DI 中

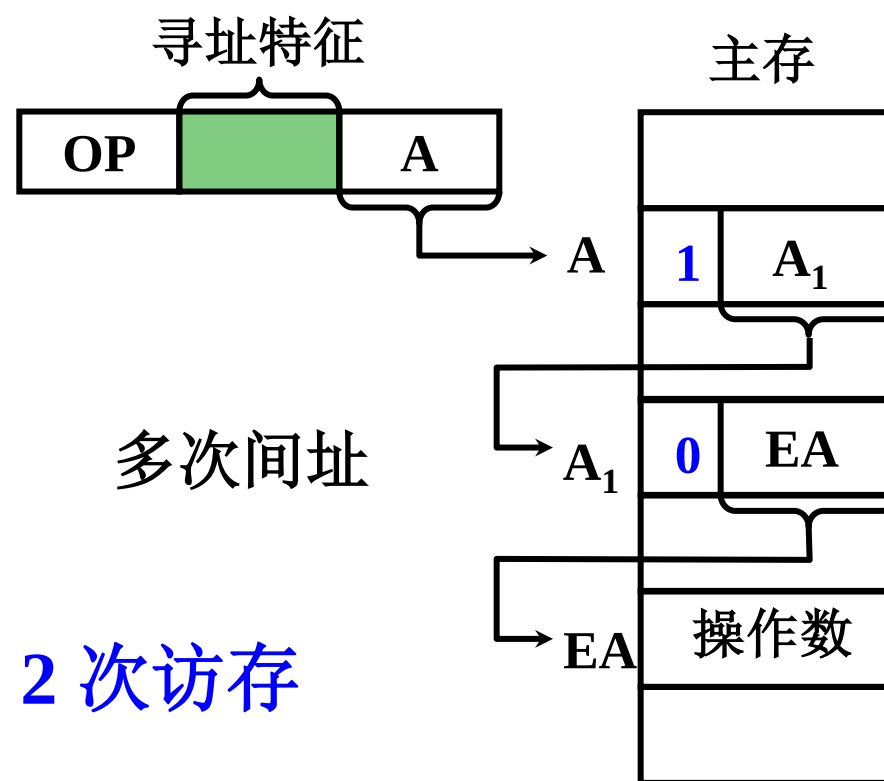
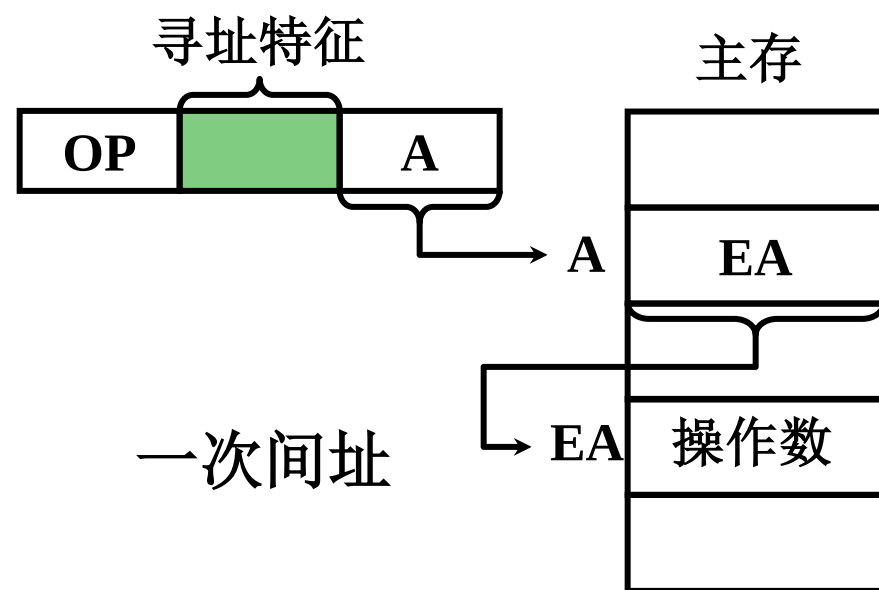
- 指令字中少了一个地址字段，可缩短指令字长

□ 数据寻址

4. 间接寻址

$$EA = (A)$$

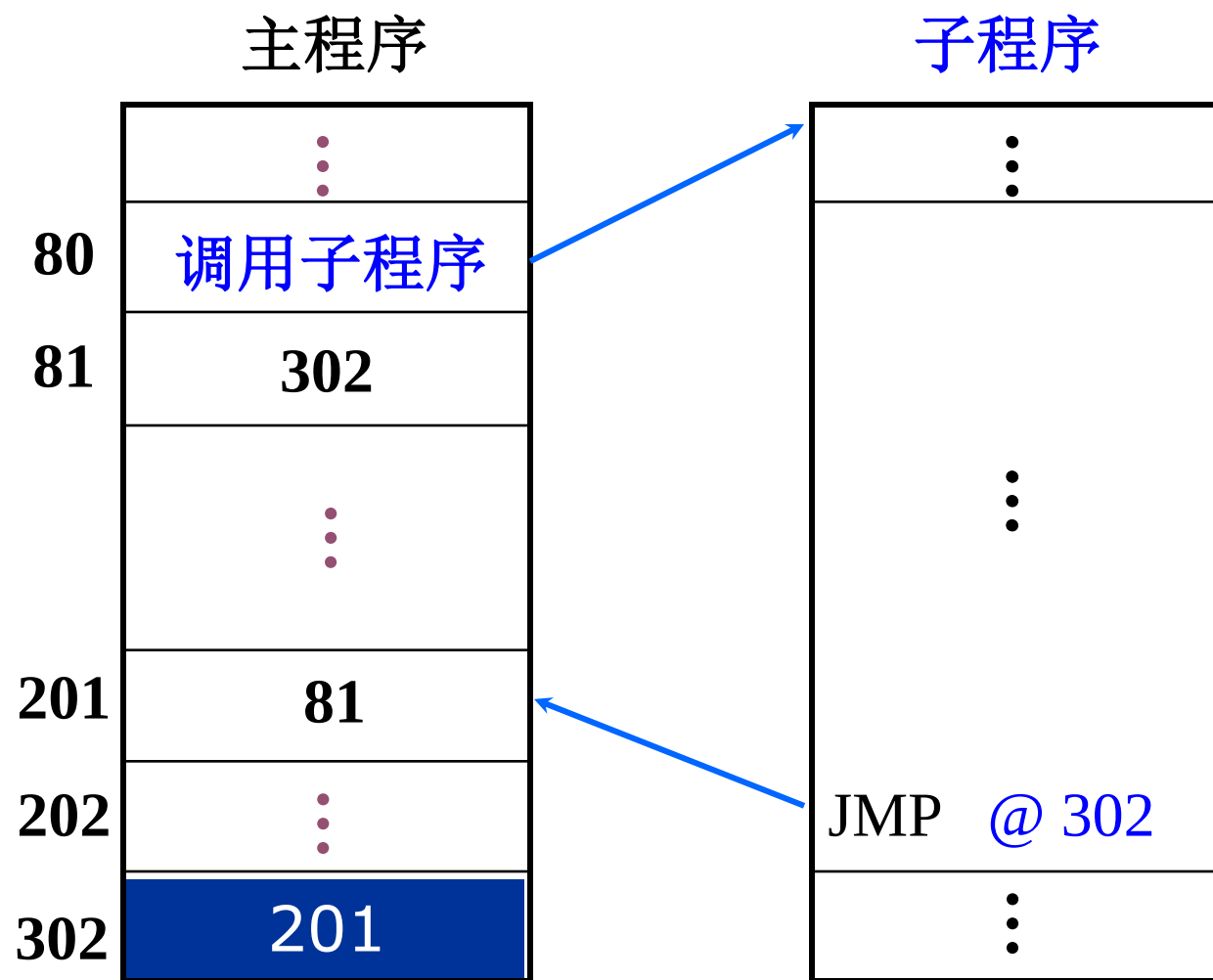
有效地址由形式地址间接提供



- 执行指令阶段 2 次访存
- 可扩大寻址范围 多次访存

□ 数据寻址

4. 间接寻址 举例

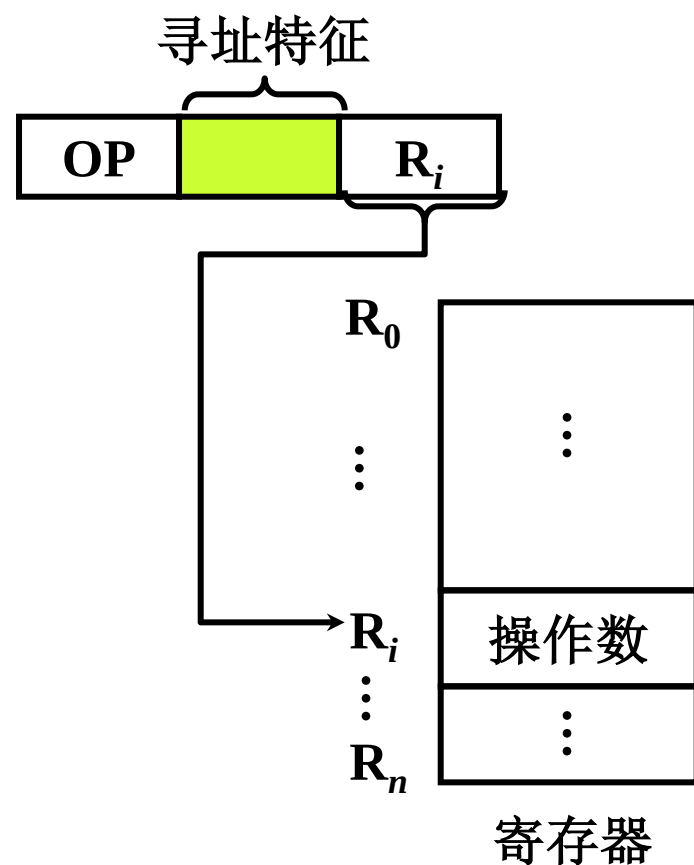


执行 **JMP @ 302**
后，程序跳转到
哪儿行开始执行？

@ 间址特征

□ 数据寻址

5. 寄存器寻址



- $EA = R_i$ 有效地址即为寄存器编号
- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长

ADD R0,R1

表示 $R0 \leftarrow R0 + R1$

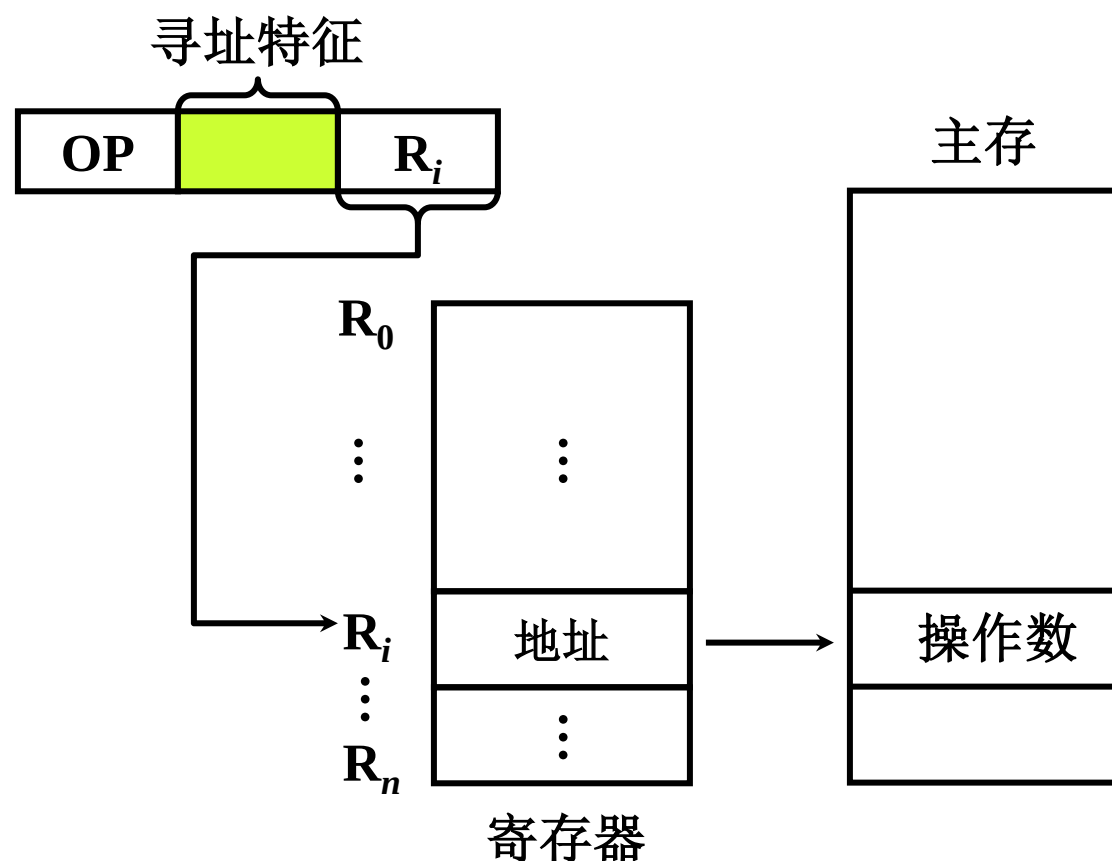
R代表寄存器寻址

MVRR R0,R1

把寄存器R1的内容传送到寄存器R0

□ 数据寻址

6. 寄存器间接寻址



- $EA = (R_i)$ 有效地址在寄存器中

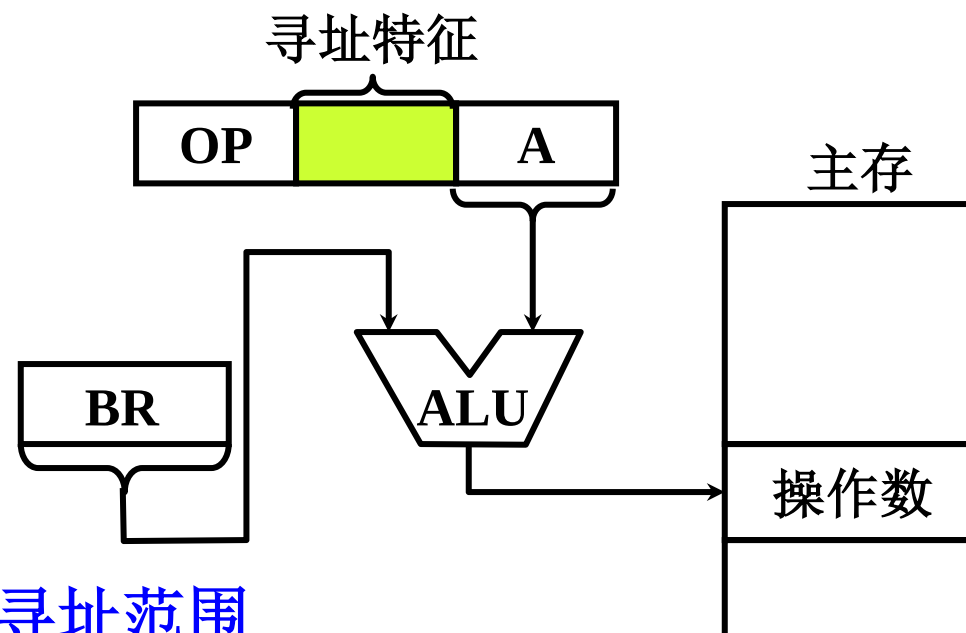
STRR [R8],R9

把R9的内容传送到以寄存器R8的内容为地址的内存单元之中；
R字母两侧加上方括号，代表寄存器间接寻址

□ 数据寻址

7. 基址寻址 (1) 采用专用寄存器作基址寄存器

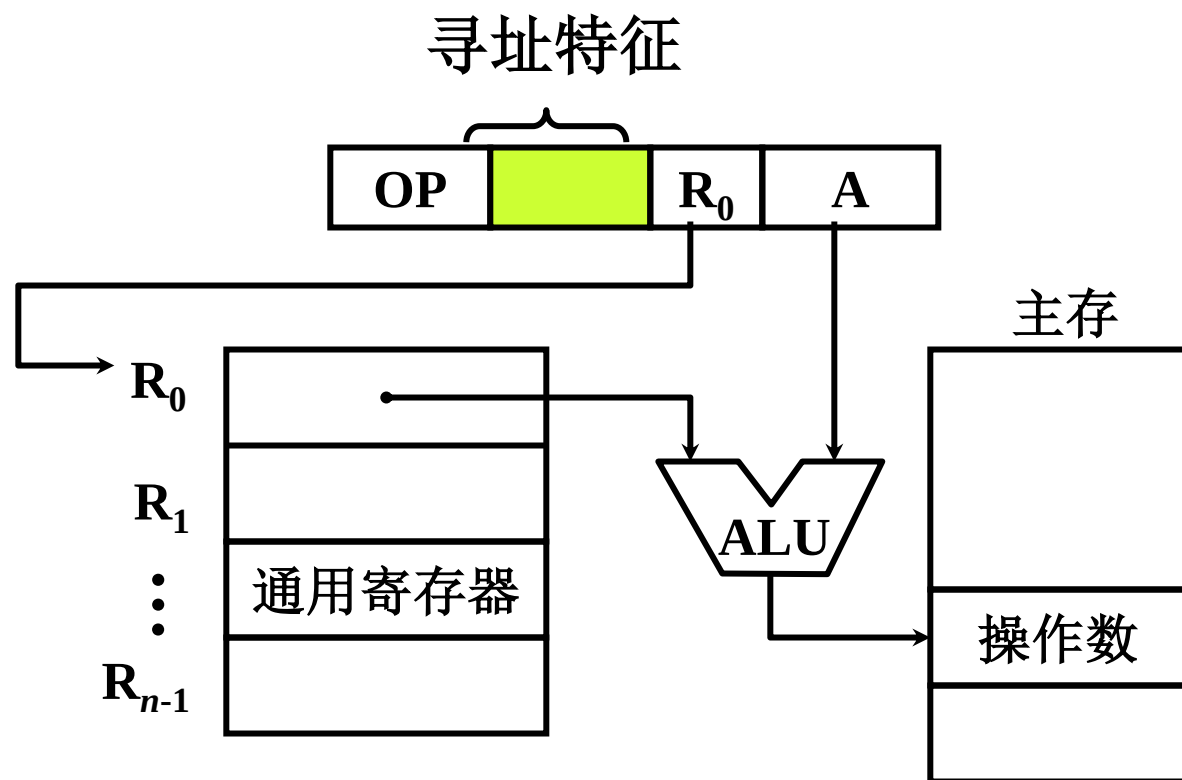
$$EA = (BR) + A \quad BR \text{ 为基址寄存器}$$



- 可扩大寻址范围
- 有利于多道程序
- **BR** 内容由操作系统或管理程序确定

□ 数据寻址

7. 基址寻址 (2) 采用通用寄存器作基址寄存器



R₀ 作基址寄存器

- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容由操作系统确定
- 在程序的执行过程中 R₀ 内容不变, 形式地址 A 可变

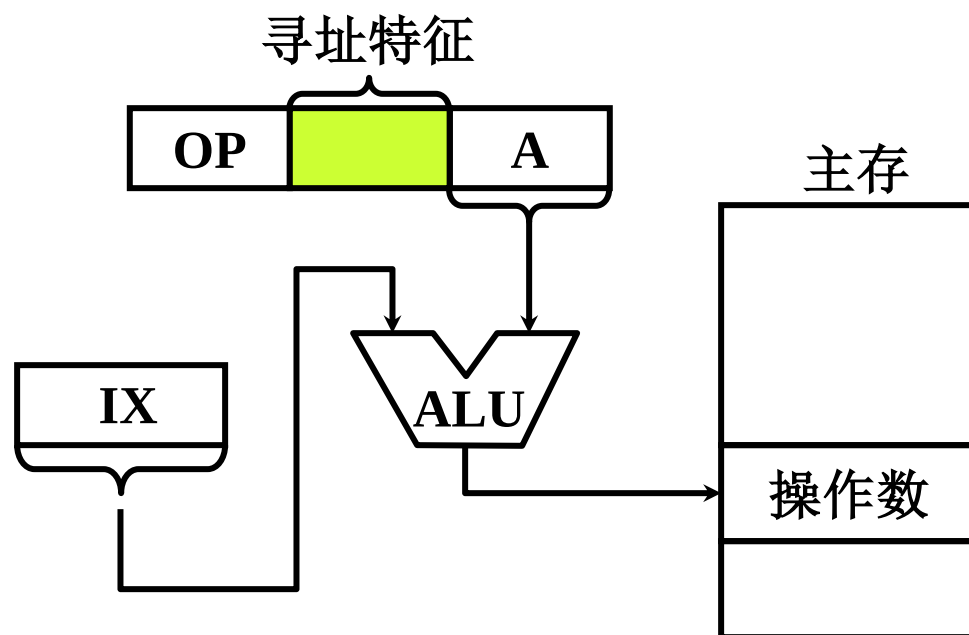
□ 数据寻址

8. 变址寻址

$$EA = (IX) + A$$

IX 为变址寄存器（专用）

通用寄存器也可以作为变址寄存器



- 可扩大寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题

LDRX R1,12[R2]

用X表示变址寻址，R2是变址寄存器

把变址寄存器R2的内容与变址偏移量12相加作为内存地址，读出的数据传送的寄存器R1

□数据寻址

8. 变址寻址 举例：设数据块首地址为 D ，求 N 个数的平均值

直接寻址

LDA D $[D] \rightarrow ACC$

ADD $D + 1$

ADD $D + 2$

⋮

ADD $D + (N - 1)$

DIV $\# N$

STA ANS 共 $N + 2$ 条指令

变址寻址

LDA $\# 0$

LDX $\# 0$ X 为变址寄存器

ADD $D[X]$ D 为形式地址

INX $(X) + 1 \rightarrow X$

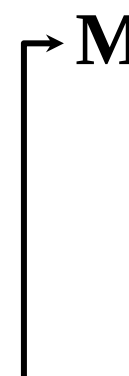
CPX $\# N$ (X) 和 $\# N$ 比较

BNE M 结果不为零则转

DIV $\# N$

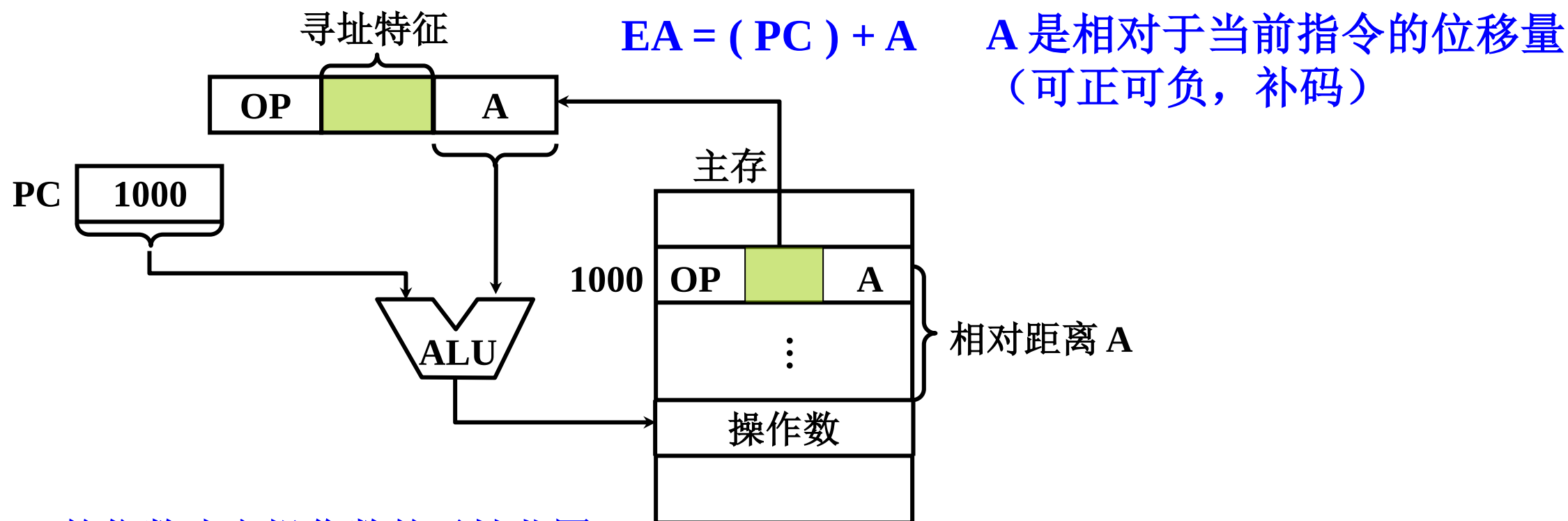
STA ANS

共 8 条指令



□ 数据寻址

9. 相对寻址



- A 的位数决定操作数的寻址范围
- 广泛用于转移指令

□ 数据寻址

9. 相对寻址 举例

	LDA	# 0	
	LDX	# 0	
	ADD	D[X]	
→ M	INX		
M+1	CPX	# N	
M+2	BNE	M → * - 3	
M+3	DIV	# N	* 相对寻址特征
	STA	ANS	

优点:

M 随程序所在存储空间的位置不同而不同

而指令 **BNE * - 3** 与 指令 **ADD D[X]** 相对位移量不变

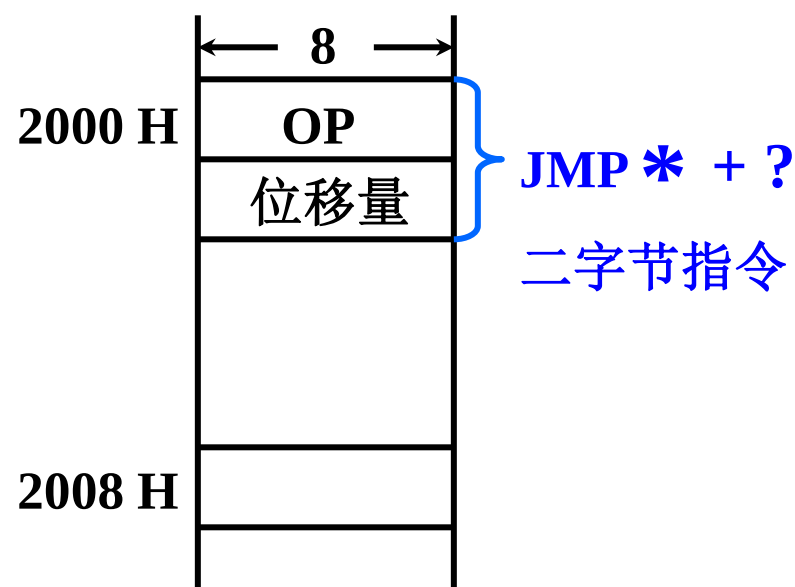
指令 **BNE * - 3** 操作数的有效地址为

$$EA = (M + 3) - 3 = M$$

□ 数据寻址

9. 相对寻址

举例：设相对寻址的转移指令占两个字节，第一个字节是操作码，第二个字节是相对位移量，用补码表示CPU每取出一个字节便自动完成 $(PC) + 1 \rightarrow PC$ 的操作。



当前指令地址 $PC = 2000H$

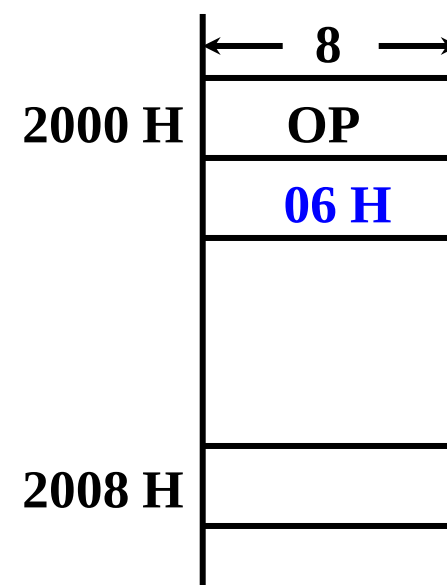
转移后的目的地址为 $2008H$

问：相对寻址的位移量是多少？

解：因为 取出 $JMP * + ?$ 后 $PC = 2002H$

故 $JMP * + ?$ 指令 的第二字节为：

$$2008H - 2002H = 06H$$



指令的寻址方式

数据寻址

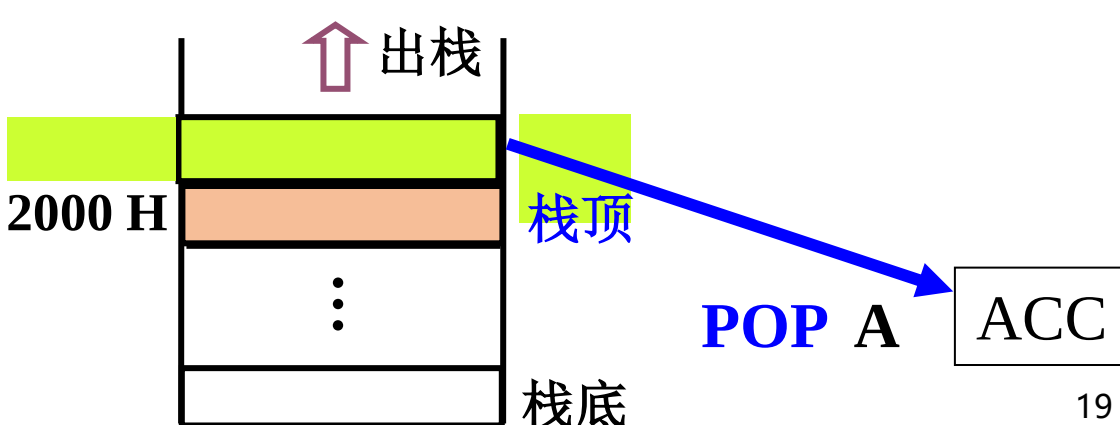
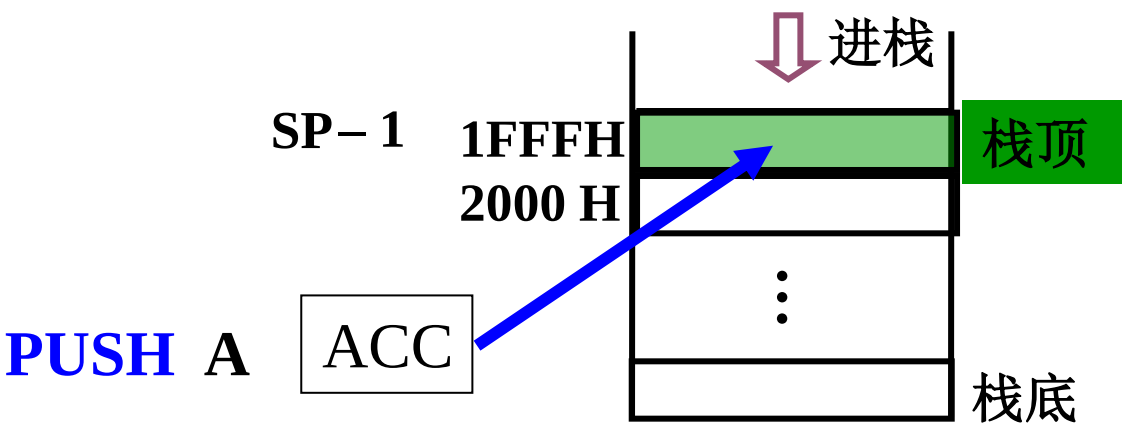
10. 堆栈寻址 (1) 特点

堆栈	{	硬堆栈	多个寄存器
		软堆栈	指定的存储空间

先进后出（一个入出口） 栈顶地址 由 **SP** 指出

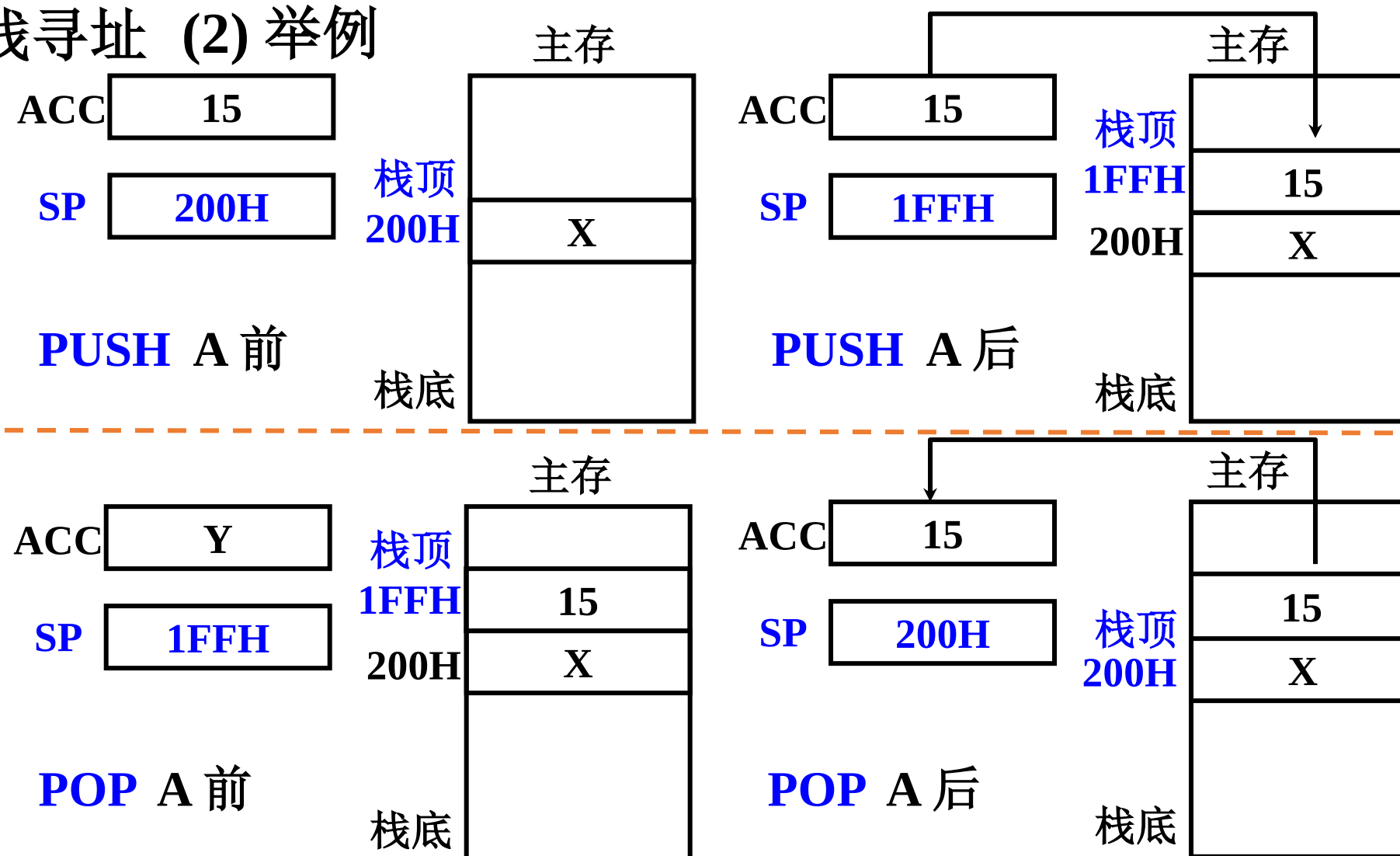
进栈 { $(SP) - 1 \rightarrow SP$
PUSH A $ACC \rightarrow (SP)$

出栈 { $(SP) \rightarrow ACC$
POP A $(SP) + 1 \rightarrow SP$



□ 数据寻址

10. 堆栈寻址 (2) 举例



□数据寻址

10. 堆栈寻址 (3) SP的修改与主存编址方法有关

① 按 字 编址

进栈 $(SP) - 1 \longrightarrow SP$

出栈 $(SP) + 1 \longrightarrow SP$

② 按 字节 编址

存储字长 16 位 进栈 $(SP) - 2 \longrightarrow SP$

出栈 $(SP) + 2 \longrightarrow SP$

存储字长 32 位 进栈 $(SP) - 4 \longrightarrow SP$

出栈 $(SP) + 4 \longrightarrow SP$

- 指令集架构的组成
- 指令的寻址方式
- CISC VS RISC

□ CISC: Complex Instruction Set Computer

- ◆ 指令集设计早期阶段倾向于：应用需要什么操作，就增加对应的指令，这导致了CISC
- ◆ 典型指令集：x86(Intel)、x86-64(AMD)

□ RISC: Reduced Instruction Set Computer

- ◆ 另一种对立的设计哲学：自然界的2-8定律
- ◆ 程序的2-8定律
 - 典型程序中 80% 的语句仅仅使用了指令集中 20% 的指令
 - 原因：执行频度高的简单指令，因复杂指令的存在，执行速度无法提高
- ◆ RISC的指导思想
 - 用 20% 的简单指令实现不常用的80%的指令功能

□ RISC的主要特征

◆ 降低访存开销

- **指令定长**：所有指令都占用32位（1个字）；
- **简化指令寻址模式**：以基地址+偏移为主；
- 只有load与store两类指令能够访存；

◆ 降低了指令执行的复杂度

- **ISA的指令不仅数量少，而且简单**；
- **把复杂留给编译**：复杂指令的功能由简单指令来实现

◆ CPU 中有多个通用寄存器，典型是32个通用寄存器如ARM、RISC-V、MIPS等

◆ 采用流水技术：一个时钟周期内完成一条指令

◆ 采用组合逻辑实现控制器

◆ 采用优化的编译程序

□ CISC的主要特征

- ◆ 系统指令复杂庞大，各种指令使用频度相差大
- ◆ 指令 长度不固定、指令格式种类多、寻址方式多
- ◆ 访存 指令 不受限制
- ◆ CPU 中设有 专用寄存器
- ◆ 大多数指令需要 多个时钟周期 执行完毕
- ◆ 采用 微程序 控制器
- ◆ 难以 用 优化编译 生成高效的代码

□ CISC与RISC的比较

- ◆ RISC更能 充分利用 VLSI 芯片的面积
- ◆ RISC 更能 提高计算机运算速度
指令数、指令格式、寻址方式少，
通用 寄存器多，采用 组合逻辑，
便于实现 指令流水
- ◆ RISC 便于设计，可降低成本，提高 可靠性
- ◆ RISC 有利于编译程序代码优化
- ◆ RISC 不易 实现 指令系统兼容

□指令集架构的组成

□指令的寻址方式

□CISC VS RISC

**作业：第7章
8、15、16题**